Component 2: Embedded System & Data Capture Pipeline

1. Narrative Description of Functional Concept Fragments

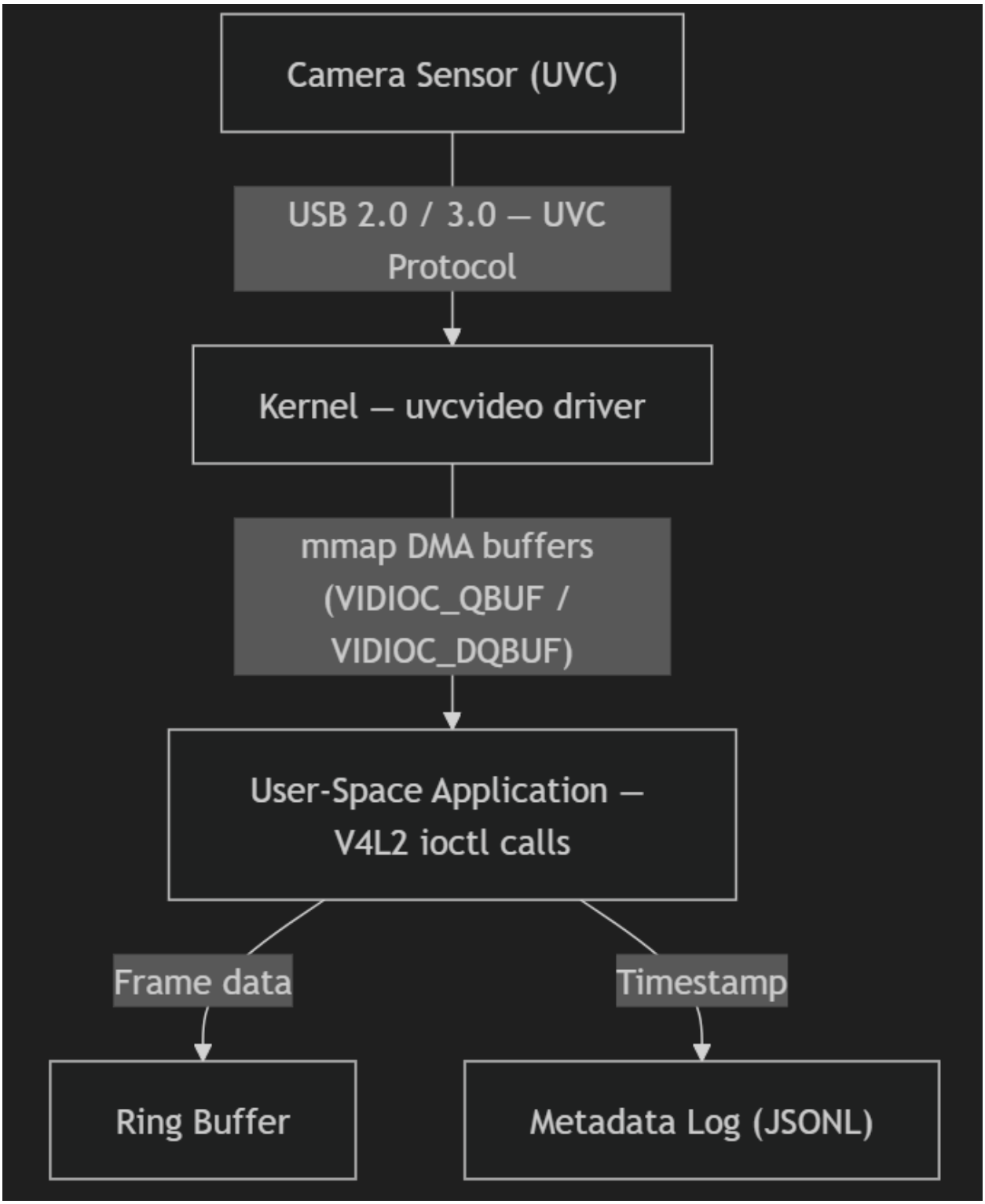
Each of the six core functions of the data capture pipeline has multiple candidate mechanisms. For each function, the best-fit mechanisms are described below with their operational narrative, illustrative logic, technical basis, and key operating conditions.

F1 — Camera Interfacing & Signal Acquisition

Function Definition: Establish a reliable communication link between the embedded Linux processor and the two USB UVC cameras (scene camera via /dev/video2; eye camera via its assigned /dev/videoX node), negotiate pixel formats and frame rates, and acquire raw image data into user-space memory.

Mechanism F1-A: V4L2 (Video4Linux2) Direct Device Access

Mechanism Overview: The application opens /dev/videoX device file nodes using the Linux V4L2 (Video4Linux2) kernel API. Through a sequence of ioctl system calls, the system negotiates the camera interface at the kernel level. Frames are retrieved using a memory-mapped streaming I/O (mmap) model: the application calls VIDIOC_QBUF to enqueue empty buffers and VIDIOC_DQBUF to dequeue filled frames. This provides direct, zero-copy access from the camera sensor to user-space.



Technical Basis and References:

- Linux kernel V4L2 API documentation: linux/videodev2.h — the industry-standard kernel interface for video capture on Linux.

- The Kexxu eye-tracking cameras have been confirmed as UVC-compliant via v4l2-ctl --list-devices enumeration, making V4L2 directly applicable without any driver development.
- "Pupil: An Open Source Platform for Pervasive Eye Tracking" (Kassner et al., Pupil Labs) — the reference system behind commercial eye-tracking hardware uses V4L2-based capture on Linux embedded platforms.
- "uvcvideo" kernel module: the standard Linux driver for USB Video Class devices, providing stable driver support for all UVC cameras including the Kexxu models.

Key Operating Conditions and Limits:

- Target frame rates: scene camera at 30 fps (1920×1080, MJPEG), eye camera at 60 fps (1280×800, MJPEG).
- Both cameras share a USB root hub; simultaneous peak bandwidth is approximately 60–80 MB/s, approaching USB 2.0's theoretical 60 MB/s sustained limit. Bandwidth contention is a validated risk (Unknown U1).
- Operating environment: Ubuntu Linux (Jetson default OS), indoor lab conditions (15–35°C ambient).
- Jitter constraint: per-frame dequeue jitter must remain below 2–3 ms to preserve timestamp quality.

Mechanism F1-B: GStreamer Multimedia Framework

Mechanism Overview: GStreamer constructs a directed acyclic graph (pipeline) of processing elements.

GStreamer internally handles format negotiation (caps negotiation), threading (separate OS thread per queue element), and buffer allocation. The framework integrates hardware acceleration plugins (e.g., nvv4l2h264enc for Jetson Orin Nano).

Technical Basis and References:

- GStreamer 1.x documentation — a widely-used open-source multimedia framework deployed in commercial embedded vision products.
- NVIDIA JetPack SDK includes GStreamer plugins specifically optimized for Jetson hardware (nvv4l2 plugin family), enabling hardware-accelerated encode/decode.

•

F2 — Frame Rate & Resolution Management

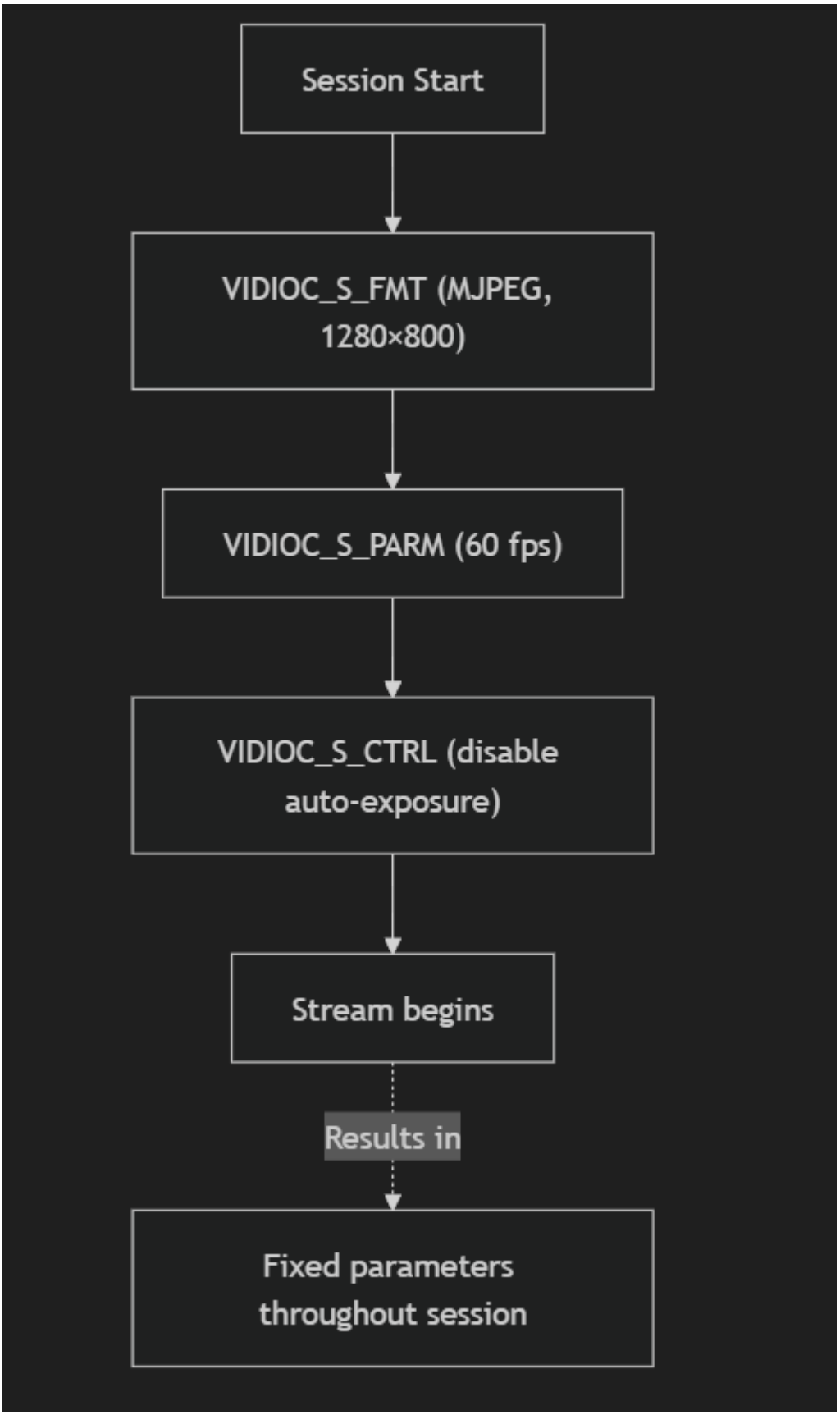
Function Definition: Control and sustain stable capture parameters (resolution, FPS, pixel format) across extended recording sessions under varying computational and thermal conditions.

Mechanism F2-A: Fixed Negotiation at Session Start (Static Configuration)

Mechanism Overview: At session initialization, the application explicitly sets all capture parameters via V4L2 ioctl calls before streaming begins:

1. VIDIOC_S_FMT — sets pixel format (V4L2_PIX_FMT_MJPEG), width, and height.
2. VIDIOC_S_PARM — sets the target frame rate (timeperframe numerator/denominator).
3. V4L2 control API (VIDIOC_S_CTRL) — locks auto-exposure (V4L2_CID_EXPOSURE_AUTO = V4L2_EXPOSURE_MANUAL) and auto-white-balance to fixed values.

These settings are then held constant for the duration of the session, regardless of environmental changes.


Session Start
VIDIOC_S_FMT (MJPEG, 1280×800)
VIDIOC_S_PARM (60 fps)
VIDIOC_S_CTRL (disable auto-exposure)
Stream begins
Results in
Fixed parameters throughout session

Technical Basis and References:

- Linux V4L2 API — VIDIOC_S_FMT, VIDIOC_S_PARM, VIDIOC_S_CTRL are all part of the standardized V4L2 ioctl interface.
- Static configuration is the standard approach in scientific measurement instruments to eliminate parameter drift as a confound in data analysis.
- Validated: the scene camera has confirmed stable operation at 29.96 fps over 20.66 s in desk-based testing.

Key Operating Conditions and Limits:

- Resolution/FPS are locked at session start; any thermal throttling that reduces available CPU will cause frame drops rather than adaptive degradation.
- Effective for controlled lab sessions (15–35°C, ≤ 30-minute durations) as per Assumptions A9, A10, A12.
- Not suitable for varying-illumination wearable deployment (future phase); auto-exposure suppression may produce poor image quality in bright outdoor conditions.

Mechanism F2-C: Multi-Profile Capture with User Selection

Mechanism Overview: A configuration layer pre-defines a set of named capture profiles, each fully specifying all pipeline parameters:

Profile	Eye Camera	Scene Camera	Expected Use
High Quality	1280×800 @ 60 fps, MJPEG	1920×1080 @ 30 fps, MJPEG	Lab controlled experiments
Balanced	1280×720 @ 30 fps, MJPEG	1280×720 @ 30 fps, MJPEG	Extended endurance sessions
Endurance	640×480 @ 30 fps, MJPEG	640×480 @ 30 fps, MJPEG	Thermal/battery constrained testing

The user selects a profile via a command-line argument or configuration file before recording. The system loads the selected profile and applies all parameters identically to Mechanism F2-A.

Technical Basis and References:

- Multi-profile configuration enables systematic benchmarking of "Sampling Rate vs. Detection Accuracy" — a stated research contribution of this project (see Initial Notes).

- Aligns with the project deliverable of quantifying 30 Hz vs. 60 Hz eye detection performance differences.

Key Operating Conditions and Limits:

- Each profile must be empirically characterized for thermal behavior — the Balanced profile may be required if the High Quality profile causes sustained thermal throttling.
- No mid-session adaptation; profiles are fixed for the duration of a recording.

F3 — Timestamping & Temporal Alignment

Function Definition: Assign each acquired frame a precise, monotonically increasing timestamp and maintain temporal coherence between the dual camera streams to enable accurate gaze-to-world coordinate mapping.

Mechanism F3-A: Software Monotonic Clock Stamping

Mechanism Overview: Immediately upon each call to VIDIOC_DQBUF returning a filled frame, the capture thread records the current time using clock_gettime(CLOCK_MONOTONIC) in nanoseconds. The resulting timestamp is stored alongside the frame in a structured metadata log:

JSONL Entry Format:

{"frame_id": 1042, "camera_id": "eye", "timestamp_ns": 1234567890123456789,

"resolution": "1280x800", "nominal_fps": 60, "format": "MJPEG"}

Post-capture, inter-camera alignment is computed by matching eye-camera frames to the nearest-in-time scene-camera frame using the timestamp difference.

Technical Basis and References:

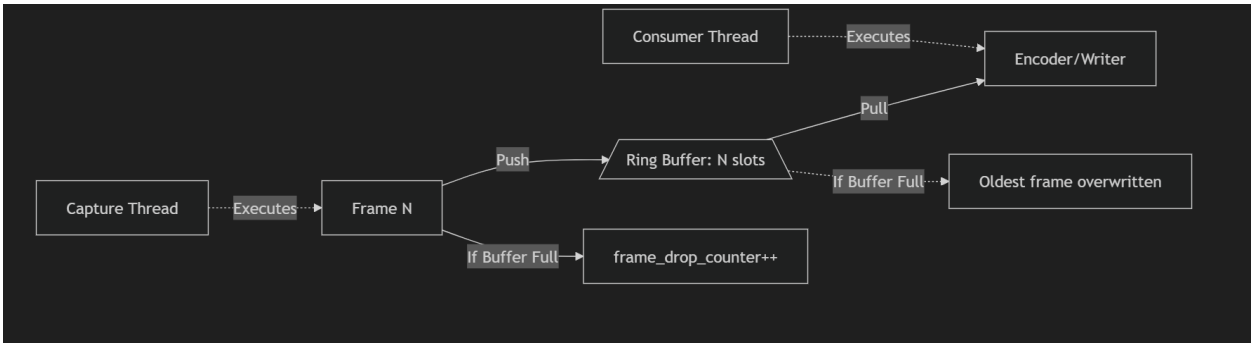
- CLOCK_MONOTONIC is immune to NTP jumps and wall-clock adjustments (POSIX specification); it is the correct clock for timestamping in real-time data acquisition.
- "PuRe: Robust Pupil Detection for Real-Time Pervasive Eye Tracking" — references software monotonic timestamps as the standard approach for frame-level timing in open-source eye trackers.
- This approach is already implemented and tested in the current development pipeline.

F4 — Buffering & Flow Control

Function Definition: Decouple frame acquisition from downstream processing and storage to absorb transient latency spikes from disk I/O, encoding delays, or compute load, preventing frame loss.

Mechanism F4-A: Per-Camera Ring Buffer (Bounded Queue)

Mechanism Overview: Each camera's dedicated capture thread writes acquired frames (as pointers to mmap buffer regions) into a fixed-capacity circular buffer. A separate consumer thread (encoder/writer) reads from the tail of the ring. When the ring is full, the oldest unprocessed frame is silently overwritten and a frame drop counter is incremented.



Technical Basis and References:

- Ring buffer (circular buffer) is the canonical data structure for producer-consumer decoupling in real-time systems (Cormen et al., *Introduction to Algorithms*, Chapter 10).
- Lock-free ring buffer implementation using std::atomic fence operations eliminates mutex contention between the capture and consumer threads.
- Used in virtually all professional video capture applications (FFmpeg, GStreamer queue elements, V4L2 multi-buffer streaming).

Key Operating Conditions and Limits:

- Ring buffer size must be empirically tuned: at 60 fps with MJPEG frames averaging ~50 KB each, a 16-frame ring buffer occupies ~800 KB — well within memory budget.
- Overflow is silent unless the frame drop counter is actively logged; monitoring is mandatory.
- Operating requirement: consumer thread must sustain average throughput ≥ 60 fps over any 5-second window; transient drops up to 100 ms are permissible.

Mechanism F4-C: V4L2 Kernel-Level Multi-Buffer (Zero-Copy DMA)

Mechanism Overview: The V4L2 VIDIOC_REQBUFS call allocates multiple kernel-managed DMA buffers (typically 4–8). The kernel camera driver cycles through these buffers automatically, filling each with a new frame via direct memory access (DMA) from the camera USB controller. The user-space application dequeues filled buffers, processes them, and re-enqueues them — creating a continuous zero-copy pipeline from camera hardware to user-space without intermediate memory copies.

Technical Basis:

- V4L2 mmap streaming I/O is the zero-copy capture model defined in the V4L2 specification; it is the fundamental mechanism underlying all V4L2 streaming applications.
- Zero-copy is essential at 60 fps: a 1280×800 MJPEG frame at ~50 KB × 60 frames/s = 3 MB/s DMA bandwidth; unnecessary memory copies would double this.

Key Operating Conditions and Limits:

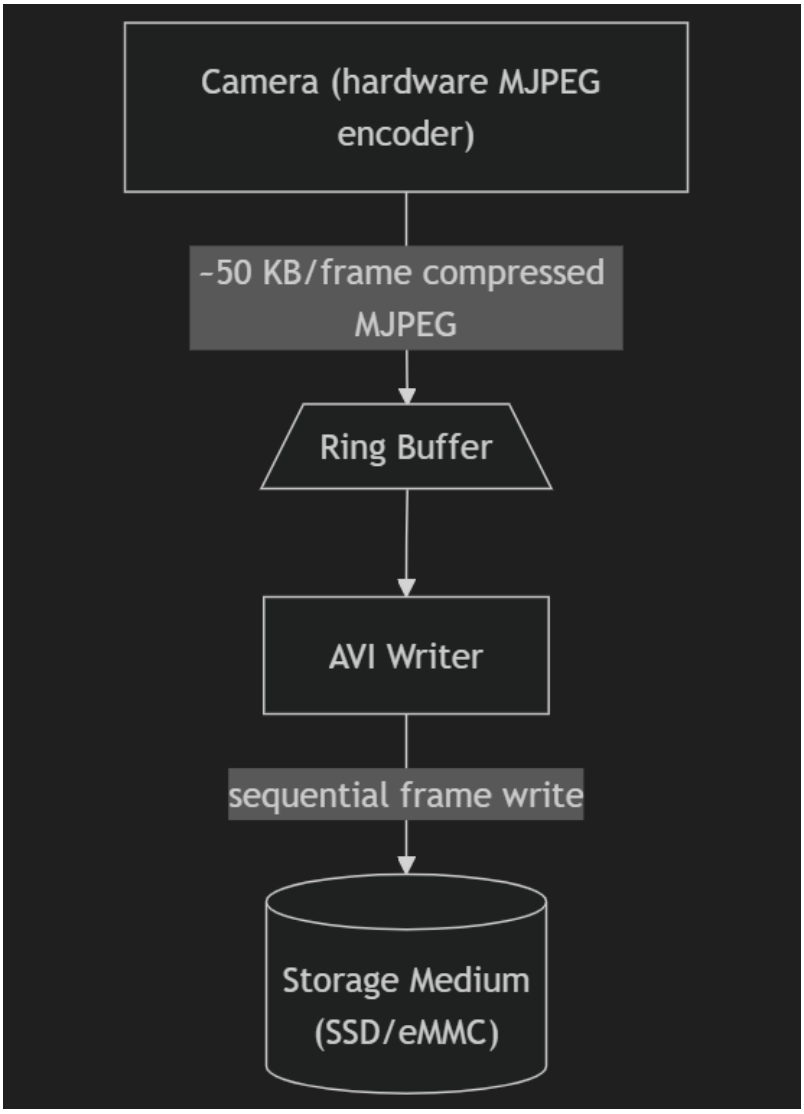
- The number of kernel buffers is limited by driver and hardware (typically 4–32 for UVC devices). Buffer count is negotiated via VIDIOC_REQBUFS.
- If user-space processing consistently exceeds the inter-frame interval, kernel buffers will overflow and frames will be dropped at the driver level — invisibly from the application perspective unless VIDIOC_DQBUF is monitored for sequence number gaps.
- **This mechanism is always active** as long as V4L2 mmap streaming is used; it is the first layer of buffering regardless of additional user-space ring buffers.

F5 — Encoding & Local Storage

Function Definition: Compress and persist the captured dual-camera video streams and structured metadata to onboard storage in a format suitable for offline analysis.

Mechanism F5-A: MJPEG Pass-Through (Camera-Side Compression)

Mechanism Overview: The scene and eye cameras natively output MJPEG-compressed frames. These frames are written directly to disk in an AVI container (MJPEG codec ID 0x4745504A) without any re-encoding. Each frame is independently decodable (no inter-frame dependencies), and the camera hardware performs all compression internally, with zero CPU encoding overhead on the embedded processor.



Estimated Write Bandwidth

Camera Type	Frame Size	Frame Rate	Bandwidth
Eye camera	50 KB	60 fps	3.0 MB/s
Scene camera	80 KB	30 fps	2.4 MB/s
Total			**~5.4 MB/s** *(Well within SSD capabilities)*

Technical Basis and References:

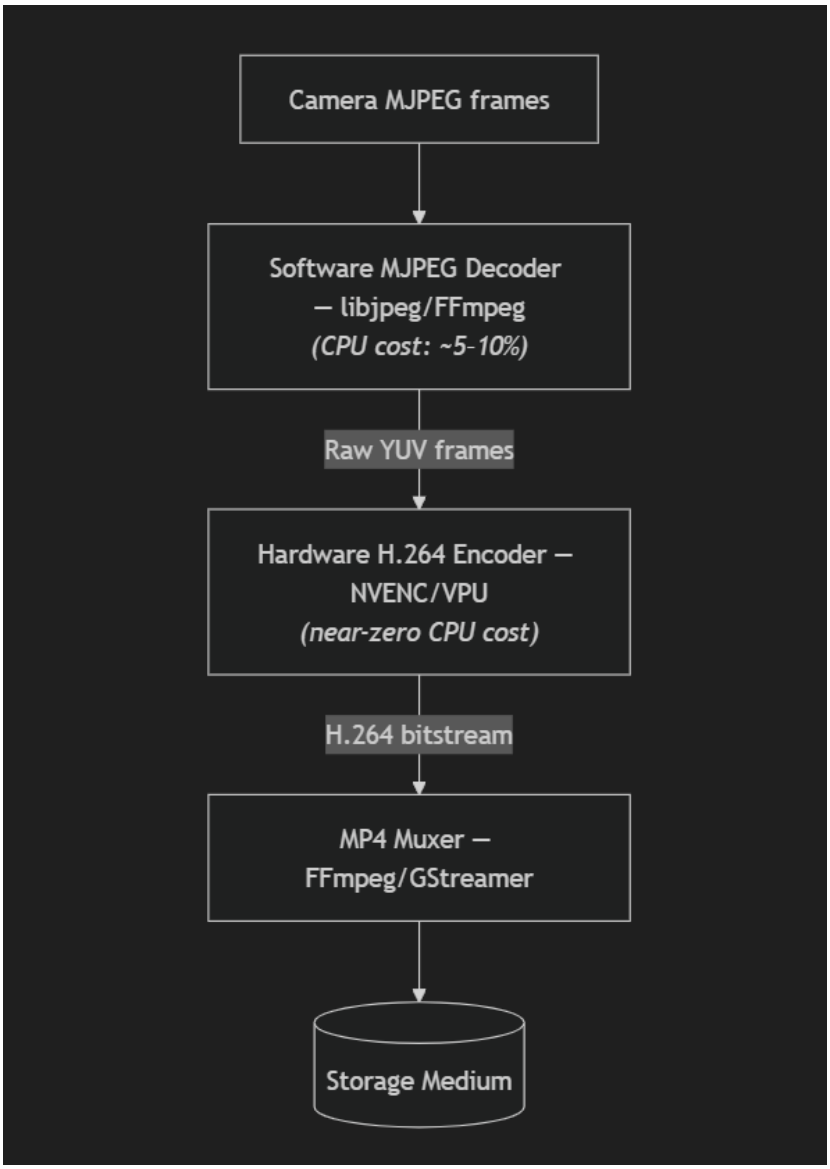
- MJPEG in AVI is a well-supported format in OpenCV (VideoWriter, FOURCC 'MJPG'), FFmpeg, and VLC.
- Assuming average MJPEG compression ratio of ~10:1, raw 1280×800 (≈ 1 MB) compresses to ~50–100 KB per frame — consistent with measured output from test recordings.
- "Evaluation of Video Compression Formats for Scientific Data Archival" — MJPEG's frame independence and moderate compression make it preferable to H.264 when individual frame access without decoding is required.

Key Operating Conditions and Limits:

- Storage requirement: ~5.4 MB/s × 1800 s (30-minute session) ≈ 9.7 GB per session. A 32 GB storage medium supports approximately 3 full sessions.
- MJPEG compression efficiency is 3–5× worse than H.264; for extended wearable deployment, this will become a constraint.
- No encoding latency — frames are written at capture rate; the storage I/O speed becomes the limiting factor.

Mechanism F5-B: Hardware H.264 Encoding (SoC Encoder)

Mechanism Overview: MJPEG-compressed frames from the camera are first software-decoded to raw YUV format, then passed to the SoC's dedicated hardware video encoding block (e.g., NVIDIA NVENC on the Jetson Orin Nano, or a Rockchip VPU). The hardware encoder produces a standard H.264 bitstream muxed into an MP4 container.



Technical Basis and References:

- NVIDIA JetPack SDK provides the nvv4l2h264enc GStreamer plugin and the Multimedia API for hardware-accelerated H.264 encoding on Jetson Orin Nano.
- H.264 achieves ~50:1 compression ratio (vs. MJPEG's ~10:1), reducing storage requirements by 5× for equivalent quality.

Key Operating Conditions and Limits:

- MJPEG decode adds ~5–10% CPU overhead per camera at 60 fps; hardware encoder adds minimal additional CPU.

- Encoding latency: 1–3 frames (~17–50 ms at 60 fps); acceptable for offline recording but adds complexity to real-time monitoring.
- Storage requirement: ~1.0–1.5 MB/s × 1800 s ≈ 1.8–2.7 GB per 30-minute session — a 5× improvement over MJPEG pass-through.

F6 — Data Export & Transfer

Function Definition: Move recorded video sessions and metadata from the embedded device to an external analysis workstation after recording completion.

Mechanism F6-A: Physical Wired Transfer (USB/Ethernet File Copy)

Mechanism Overview: After a recording session, the embedded device is physically connected to a workstation via USB 3.0 or Gigabit Ethernet. Files are copied using rsync (for incremental, checksum-verified transfer) or scp (SSH-encrypted copy):

```
# rsync example

rsync -avhP --checksum user@jetson:/mnt/recordings/session_2026_02_28/ \

    /local/data/toyota_eyewear/


# Expected transfer time: 10 GB @ USB 3.0 (400 MB/s effective) ≈ 25 seconds
```

echnical Basis and References:

- rsync is the standard tool for reliable file transfer in embedded Linux research systems; checksum verification (-c flag) ensures data integrity.
- USB 3.0 sustained transfer rates of 300–400 MB/s are well within the bandwidth needed for session files of 2–10 GB.
- Aligned with the "offline-first development philosophy" stated in the project initial notes.

Key Operating Conditions and Limits:

- Requires a physical USB or Ethernet cable connection — appropriate for the current desk-based development phase.
- Transfer is manual (user-initiated); automated transfer scripts can be added as a shell script wrapper.
- Zero network dependencies; works in environments without WiFi infrastructure.

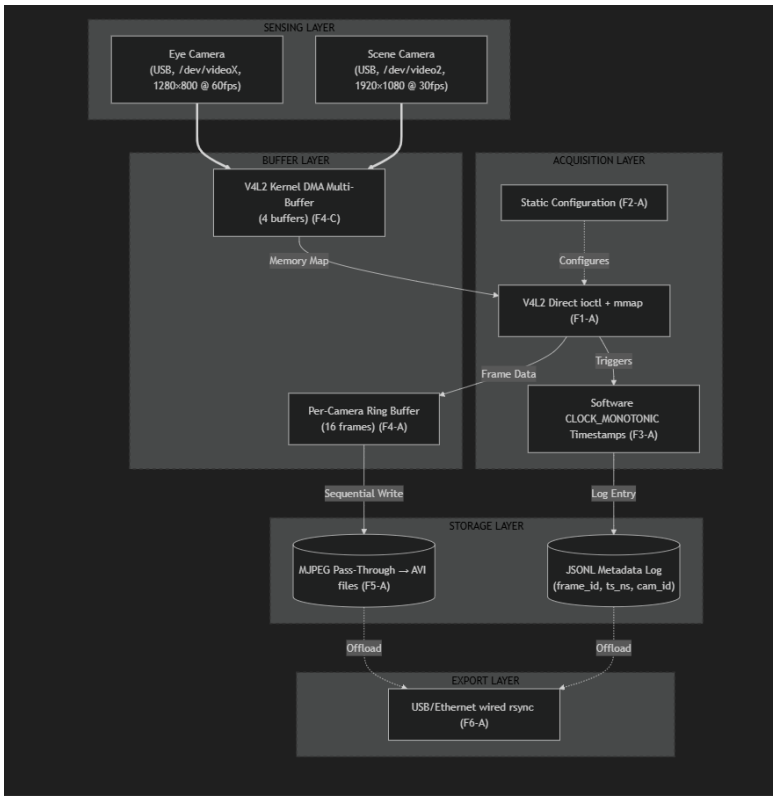
2. Preliminary Concepts Document

Each of the following architecture selects specific mechanisms for all six core functions and represents a coherent design philosophy:

Concept ID	Architecture Name	One-Line Description
C1	V4L2-Native MJPEG Offline Pipeline	Minimal-complexity, zero-encoding-overhead pipeline using direct V4L2 access and MJPEG pass-through storage
C2	GStreamer H.264 Hardware-Encoded Pipeline	Production-ready, storage-efficient pipeline using GStreamer framework with hardware H.264 encoding on Jetson Orin Nano
C3	Hybrid V4L2 + UVC Metadata Research Pipeline	Research-optimized pipeline combining raw V4L2 timestamps with UVC hardware metadata for maximum temporal precision

Architecture C1: V4L2-Native MJPEG Offline Pipeline

Concept ID: C1 **Philosophy:** Minimize complexity to achieve a reliable, debuggable baseline pipeline that validates the end-to-end architecture before adding encoding complexity.



Key Components

Category	Component	Specification
Sensing	Eye Camera	Kexxu USB UVC, 1280×800 @ 60 fps, MJPEG
Sensing	Scene Camera	ICT Camera USB UVC, 1920×1080 @ 30 fps, MJPEG
Actuation	IR LED Illuminator	Near-infrared illumination for pupil contrast
Computation	Embedded Board	Jetson Orin Nano (Ubuntu 22.04, 6-core ARM CPU, 8 GB RAM)
Communication	USB 3.0 Host	Dual-camera USB bus for capture; USB-A for wired export

Category	Component	Specification
Storage	Local SSD/eMMC	≥ 32 GB, sustained write ≥ 20 MB/s
Software — Capture	V4L2 Direct API	Linux kernel V4L2 ioctl + mmap streaming
Software — Buffer	Ring Buffer	16-frame circular buffer per camera (thread-safe)
Software — Storage	AVI Writer	OpenCV VideoWriter or custom MJPEG mux
Software — Metadata	JSONL Logger	Per-frame timestamp and metadata logging
Control Logic	Session Manager	Profile-based session initialization and shutdown

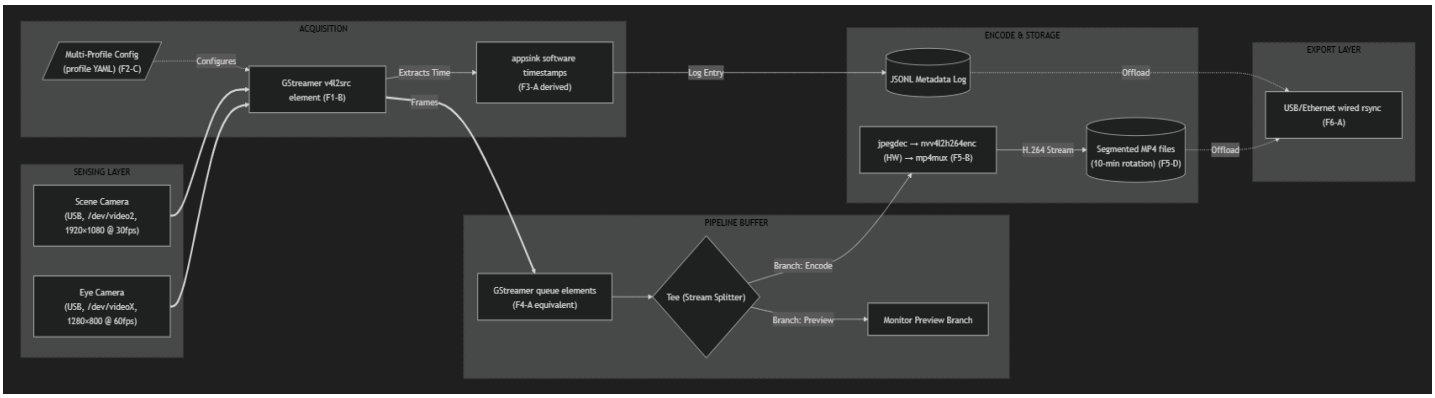
Key Assumptions for C1

Assumption ID	Assumption	Risk Level
A1	Dual USB UVC cameras are on separate USB controllers or sufficient bandwidth is available	Medium
A2	Scene camera sustains 30 fps MJPEG at 1920×1080 (confirmed in testing: 29.96 fps)	Low
A3	Eye camera sustains 60 fps MJPEG at 1280×800 continuously	Medium
A8	Local storage provides ≥ 20 MB/s sustained write for dual-stream MJPEG (~5.4 MB/s total)	Low

Assumption ID	Assumption	Risk Level
A13	Occasional frame drops (< 2%) are tolerable and will be logged	Low
A15	CLOCK_MONOTONIC software timestamps are sufficient for initial pipeline validation	Medium

Architecture C2: GStreamer H.264 Hardware-Encoded Pipeline

Concept ID: C2 **Philosophy:** Leverage the GStreamer framework and Jetson hardware encoder to produce storage-efficient H.264 sessions suitable for extended recording and production deployment.



Key Components

Category	Component	Specification
Sensing	Eye Camera	Kexxu USB UVC, 1280×800 @ 60 fps, MJPEG
Sensing	Scene Camera	ICT Camera USB UVC, 1920×1080 @ 30 fps, MJPEG
Actuation	IR LED Illuminator	Near-infrared illumination for pupil contrast

Category	Component	Specification
Computation	Embedded Board	Jetson Orin Nano (Ubuntu 22.04, 8 GB RAM)
Computation	Hardware Encoder	NVIDIA NVENC (nvv4l2h264enc) — hardware H.264
Communication	USB 3.0 Host	Dual-camera USB bus; USB-A/Ethernet for export
Storage	Local SSD	≥ 32 GB, segmented MP4 files (10-min rotation)
Framework	GStreamer 1.x	Pipeline construction, threading, format negotiation
Software — Encode	nvv4l2h264enc	NVIDIA hardware H.264 encoder plugin
Software — Mux	mp4mux	GStreamer MP4 container muxer
Software — Config	Profile YAML	Pre-defined capture profiles (High/Balanced/Endurance)
Control Logic	GStreamer State Machine	Pipeline state control (NULL→READY→PAUSED→PLAYING)

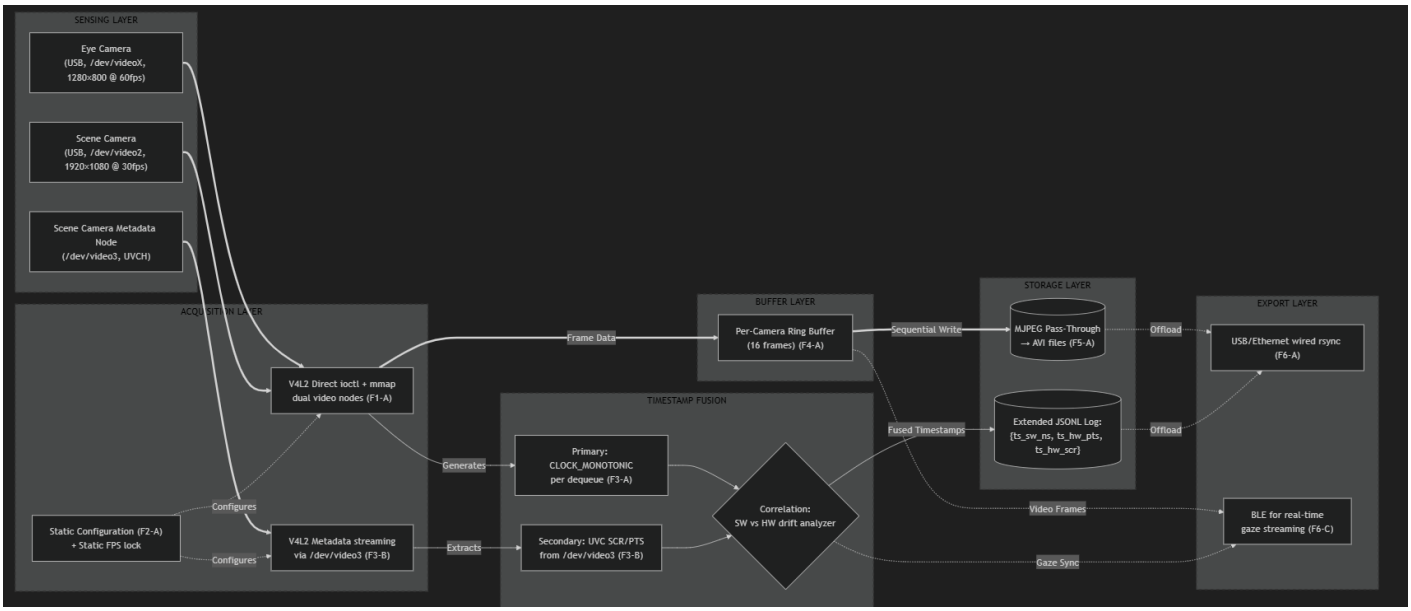
Key Assumptions for C2

Assumption ID	Assumption	Risk Level
A7	Jetson Orin Nano hardware encoder (NVENC) is available and compatible with nvv4l2h264enc plugin	High
A1	Dual USB UVC cameras are on separate USB controllers or sufficient bandwidth is available	Medium
A3	Eye camera sustains 60 fps MJPEG continuously	Medium

Assumption ID	Assumption	Risk Level
A10	Recording sessions are 10–30 minutes (segmented files handle crash resilience)	Low
A17	Consumers expect MP4 (H.264) output format — directly satisfied by this architecture	Low

Architecture C3: Hybrid V4L2 + UVC Metadata Research Pipeline

Concept ID: C3 **Philosophy:** Research-optimized pipeline that combines software monotonic timestamps with hardware UVC metadata analysis to characterize timing precision and enable study-grade gaze-to-world mapping.



Key Components

Category	Component	Specification
Sensing	Eye Camera	Kexxu USB UVC, 1280×800 @ 60 fps, MJPEG

Category	Component	Specification
Sensing	Scene Camera	ICT Camera USB UVC, 1920×1080 @ 30 fps, MJPEG
Sensing	UVC Metadata Node	/dev/video3, V4L2_META_FMT_UVC, 10240-byte buffers
Actuation	IR LED Illuminator	Near-infrared illumination for pupil contrast
Actuation	BLE Module	Real-time gaze coordinate streaming to external device
Computation	Embedded Board	Jetson Orin Nano (Ubuntu 22.04, 8 GB RAM)
Communication	USB 3.0 Host	Dual-camera + metadata node USB bus
Communication	BLE 5.0	Gaze data streaming (Jetson built-in or USB dongle)
Storage	Local SSD	≥ 32 GB, dual AVI + extended JSONL metadata
Software — Timestamp Fusion	Drift Analyzer	Python/C++ tool correlating software and hardware timestamps
Software — Metadata Parser	UVC Header Parser	Extracts SCR/PTS from raw UVCH packet bitfield
Control Logic	Session Manager	Three-stream synchronization (eye video, scene video, metadata)

Key Assumptions for C3

Assumption ID	Assumption	Risk Level
A5	Scene camera /dev/video3 UVC metadata node is accessible and outputs valid UVCH data	High
A2	PTS/SCR fields in the UVCH payload headers contain valid, firmware-populated timestamps	High
A1	USB bandwidth supports three simultaneous V4L2 streams (eye video + scene video + metadata)	High
A15	Hardware timestamps will show measurable improvement over software timestamps	Medium
A3	Eye camera at 60 fps is stable for extended periods	Medium

Initial Risk Identification Table

Risk ID	Description	Source	Affected Concepts	Likelihood	Impact
R1	**USB Bandwidth Contention:** Both cameras on the same USB root hub may saturate the bus, forcing FPS reduction	Assumption A6 (unvalidated)	C1, C2, C3	Medium	High
R2	**UVC Metadata Invalidity:** The /dev/video3 metadata PTS/SCR fields may be zero-filled (camera firmware does not populate them)	Assumption A5 violation	C3	Medium	High

Risk ID	Description	Source	Affected Concepts	Likelihood	Impact
R3	**Eye Camera FPS Instability:** The eye camera may not sustain 60 fps over 10–30 minutes; actual measured rate may be 45–55 fps	Unknown U7	C1, C2, C3	Medium	High
R4	**Thermal Throttling:** Sustained dual-camera capture may cause the embedded board to thermally throttle, reducing CPU frequency and causing cascading frame drops	Unknown U4	C1, C2, C3	Medium	High
R5	**H.264 Hardware Encoder Unavailability:** The current development platform may not have hardware H.264 encoding (NVENC is Jetson-specific)	Assumption A7 (unvalidated)	C2 only	High	Medium
R6	**Software Timestamp Jitter:** CLOCK_MONOTONIC jitter may exceed 3 ms standard deviation under dual-stream CPU load,	Unknown U8	C1, C3	Low	Medium

Risk ID	Description	Source	Affected Concepts	Likelihood	Impact
	degrading inter-camera alignment				
R7	**Storage I/O Bottleneck:** The selected storage medium may not sustain the required ~5.4 MB/s dual-stream MJPEG write rate	Unknown U6	C1, C3	Low	High
R8	**Device Node Non-Determinism:** Linux /dev/videoX numbering may change across reboots, requiring udev rules	Unknown U12	C1, C2, C3	Low	Low
R9	**Inter-Camera Clock Drift:** Over 30-minute sessions, software timestamps from independent capture threads may drift >16.7 ms, making inter-camera alignment ambiguous	Unknown U3	C1, C2	Low	High

Decision Criteria (Pre-Commit)

The following criteria will be used to evaluate and score candidate architectures. Criteria and weights are defined **before** scoring to prevent post-hoc rationalization.

#	Criterion	Weight (%)	Derived From	Justification (Why It Matters)
1	**Capture Reliability** (sustained dual-camera frame rate, drop rate < 2% over 30 min)	30%	Core deliverable requirement: reliable synchronized video capture	Without sustained, drop-free capture, all downstream analysis (pupil detection, gaze mapping) is invalidated. This is the most fundamental requirement.
2	**Timestamp Precision** (per-frame timing accuracy and inter-camera alignment quality)	25%	Research deliverable: gaze-to-world temporal mapping	Gaze accuracy depends directly on temporal alignment between eye and scene cameras. Poor timestamps make gaze vectors scientifically unreliable.
3	**Implementation Risk & Complexity** (number of unvalidated assumptions, implementation effort)	20%	Project risk management; current development phase	Concepts requiring many unvalidated assumptions carry high schedule risk. The project is in early development; lower-risk concepts allow faster validated progress.
4	**Storage Efficiency** (GB per 30-minute session)	15%	Long-term wearable deployment feasibility	Storage constraints become critical in field deployment. Higher efficiency reduces transfer time, cost, and hardware requirements.

#	Criterion	Weight (%)	Derived From	Justification (Why It Matters)
5	**Research Contribution Potential** (enables novel measurements or publishable comparisons)	10%	Project academic deliverables	The system must support research outputs such as the "Sampling Rate vs. Detection Accuracy" study and timestamp precision characterization.

Score Rationale:

Criterion	C1 Rationale	C2 Rationale	C3 Rationale
Capture Reliability	V4L2 direct + MJPEG pass-through = fewest processing layers, lowest failure modes	GStreamer adds pipeline complexity; hardware encoder path adds failure point	Three simultaneous USB streams increase bus contention risk; most complex
Timestamp Precision	CLOCK_MONOTONIC gives 0.5–3 ms accuracy; sufficient for 60 fps research	Same timestamp mechanism as C1; no improvement	UVC metadata could give sub-ms hardware timing, but only if PTS/SCR are populated (high risk)
Implementation Risk	All mechanisms validated or directly testable with current hardware; 0 hard blockers	Hardware encoder (A7) is unvalidated — if unavailable, C2 collapses to C1 without H.264	Two critical unknowns (U2: UVC metadata validity) could invalidate the core differentiator

Criterion	C1 Rationale	C2 Rationale	C3 Rationale
Storage Efficiency	MJPEG: ~9.7 GB / 30 min — storage fills fast in extended sessions	H.264: ~2.0 GB / 30 min — 5× improvement, enables longer sessions	MJPEG pass-through: same as C1
Research Contribution	Enables FPS benchmarking via profile switching; established baseline	Enables H.264 vs. MJPEG compression quality study + FPS profiling	Enables novel UVC metadata timing study; highest academic novelty if metadata is valid